ĐẠI HỌC QUỐC GIA TP HCM

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO GIỮA KỲ

Môn: **Hệ CSDL tiên tiến**

Thực hiện **Nhóm 1**:

PHẠM THỊ THÚY NGỌC – 250104046

PHẠM THÚY VI – 250104058

CUNG PHƯƠNG ANH – 250104029

PHẠM THỊ THU VÂN - 250104057

Giảng viên**: PGS.TS NGUYỄN ĐÌNH THUÂN**

TP HỒ CHÍ MINH – NĂM 2026

Bảng phân công công việc

Họ và tên	MSSV	Bảng phân công
Phạm Thị Thuý Ngọc	250104046	Nội dung chính (Đặt vấn đề & 4 Lỗ hổng kỹ thuật)
Phạm Thuý Vi	250104058	Code & Tái lập Thực nghiệm
Cung Phương Anh	250104029	Hướng phát triển từ bài báo
Phạm Thị Thu Vân	250104057	Phân tích tính SOTA của bài báo

BÁO CÁO GIỮA KỲ.....1

1. MỞ ĐẦU, CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH PIPELINE ML CỦA LQO 3
1.1. Mở đầu 3
1.2. Cơ sở lý thuyết 4
1.2.1. Bộ tối ưu truyền thống (Hình 1, nửa trên) 4
1.2.2. Bộ tối ưu học máy – LQO (Hình 1, nửa dưới) 4
1.2.3 Khái niệm ML cốt lõi 5
1.3. Phân tích 4 lỗ hổng trong pipeline ml 5
1.3.1. Lỗ hổng (1): Sinh dữ liệu huấn luyện (§3) 5
1.3.2. Lỗ hổng (2): Mã hóa truy vấn/kế hoạch (§4) 6
1.3.3. Lỗ hổng (3): Huấn luyện mô hình (§5) 6
1.3.4. Lỗ hổng (4): Đánh giá và rò rỉ dữ liệu (§6, §7.2) 7
2. Tính SOTA của bài báo 8
3. Phát triển từ bài báo (What, How, Why) 9
3.1. WHAT – Định hướng nghiên cứu 9
3.1.1. Cơ sở hình thành hướng phát triển 9
3.1.2. Mục tiêu nghiên cứu 9
3.1.3. Phạm vi của đề xuất 9
3.2. HOW – Khuyến nghị và giải pháp kỹ thuật 9
3.2.1. Kiến trúc tổng quan được đề xuất 10
3.2.2. Biểu diễn đồng thời Query và Plan 10
3.2.3. Xếp hạng kế hoạch bằng Learning-to-Rank 10
3.2.4. Phân chia dữ liệu bằng Base Query Sampling 11
3.2.5. Chuẩn hóa điều kiện đo: Hot Cache và DB Tuning 11
3.2.6. Kế hoạch kiểm chứng đề xuất 11
3.3. WHY – Cơ sở và ý nghĩa của hướng phát triển 12
3.3.1. Tối ưu hóa phải xét toàn bộ chi phí xử lý 12
3.3.2. Khả năng tổng quát hóa quyết định tính hữu dụng 12
3.3.3. So sánh công bằng là điều kiện để kết luận có giá trị 13
3.4. Kết luận và hướng nghiên cứu tiếp theo 13
4. Kế hoạch thực nghiệm và mở rộng 13
4.1. Mục tiêu 13

4.2. Hệ quản trị CSDL và môi trường 14
4.3. Phương pháp chạy lại 14
4.4. Metric đo câu SQL 15
4.5. Chạy plan và lưu log 15
4.6. Phân loại plan bằng data mining 16
4.7. Thêm benchmark 18
4.8. Kết quả sơ bộ trên log của tác giả 18

1. MỞ ĐẦU, CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH PIPELINE ML CỦA LQO

1.1. Mở đầu

Bối cảnh: Với một câu SQL, DBMS phải chọn một trong rất nhiều kế hoạch thực thi tương đương về kết quả nhưng khác xa nhau về thời gian chạy. Khi đồ thị join là clique gồm *n* bảng, số kế hoạch logic trong trường hợp xấu nhất là O(n!), và bài toán được xếp vào lớp NP-khó [37]. Vì vậy DBMS không thể vét cạn với truy vấn lớn mà phải dùng

heuristic. Ví dụ, PostgreSQL bỏ phương pháp vét cạn khi truy vấn có từ 12 bảng trở lên (§1, chú thích 1).

Sự ra đời của LQO: Nhờ tiến bộ của học sâu, và vì bài toán tối ưu truy vấn (QOP) dễ phát biểu lại thành bài toán ML, nhiều bộ tối ưu truy vấn học máy (LQO) đã ra đời: Neo [23], Bao [22], Balsa [39], Lero [44], LEON [4], LOGER [3], HybridQO [41], RTOS [42] (Abstract, §2).

Vấn đề: Mỗi LQO tự chọn cách sinh dữ liệu, cách chia train/test và cấu hình DBMS riêng, đồng thời vi phạm các nguyên tắc ML cơ bản [30]. Do đó kết quả của các LQO không so sánh công bằng được với nhau (§1). Ví dụ, RAM dùng trong thực nghiệm dao động từ 15 GB (Bao) đến 512 GB (Lero) (Bảng 2).

Câu hỏi nghiên cứu. Khi được đánh giá công bằng và end-to-end, LQO có thật sự tốt hơn PostgreSQL không?

Cách tiếp cận và đóng góp: Bài báo xem LQO như một pipeline ML gồm 4 giai đoạn và phân tích lỗi ở từng giai đoạn (§1, Hình 1):

Giai đoạn	**Đóng góp**	**Mục**
(1) Sinh dữ liệu huấn luyện	Chỉ ra các hạn chế khi sinh dữ liệu	§3
(2) Mã hóa truy vấn/kế hoạch	Chẩn đoán lỗi bất biến	§4
(3) Huấn luyện	Đề xuất cách huấn luyện ổn định hơn	§5
(4) Đánh giá	Thống nhất cách chia dữ liệu	§6, §7.2
Đóng góp thêm	Framework benchmark end-to-end và đánh giá 5 LQO	§7, §8

Kết luận: "current LQOs do not systematically perform better than PostgreSQL" (§1). Thứ hạng chung là PostgreSQL > HybridQO > Bao > Neo > Balsa > LEON (§8.2.1).

1.2. Cơ sở lý thuyết

1.2.1. Bộ tối ưu truyền thống (Hình 1, nửa trên)

- SQL → kế hoạch logic (thứ tự join) → kế hoạch vật lý (thuật toán join: hash, merge, nested loop; cách scan) → tìm kiếm bằng quy hoạch động, hoặc thuật toán di truyền

GEQO khi có từ 12 bảng [34], [28]. Việc tìm kiếm dựa trên cost model, và cost model dựa trên ước lượng cardinality (§1, §7.1).

1.2.2. Bộ tối ưu học máy – LQO (Hình 1, nửa dưới)

- LQO thay khối biểu diễn và ra quyết định của bộ tối ưu truyền thống bằng mã hóa truy vấn/kế hoạch và mô hình dự đoán. Các LQO chính (§2, Bảng 1):

LQO	**Ý tưởng**	**Mô hình**	**Đầu ra**
Neo [23]	LQO end-to-end đầu tiên; mạng nơ-ron dự đoán latency, tìm kiếm tham lam	Hồi quy, Tree-CNN	Plan
Bao [22]	Chọn **hint set** (bật/tắt loại join/scan) cho PostgreSQL	Hồi quy, Tree-CNN	Hint set
Balsa [39]	Giống Neo, nhưng pre-train bằng cost, dùng timeout	Hồi quy, Tree-CNN	Plan
LEON [4], Lero [44]	Learning-to-rank (LTR)	LTR, Tree-CNN	Plan
HybridQO [41]	Lấy ứng viên từ DBMS (MCTS trên cost), rồi chọn theo latency dự đoán	Hồi quy, Tree-LSTM	Plan
RTOS [42], LOGER [3]	RL + Tree-LSTM; LOGER thêm loại join	Hồi quy, Tree-LSTM	Plan / Hint

1.2.3 Khái niệm ML cốt lõi

- Dữ liệu (D, y) (§3.3): D là vector đặc trưng (với RL là bộ (s, a, s′)); y là latency, cost hoặc ranking. Nguyên tắc bất biến [17]: cùng đầu vào thì phải cùng đầu ra (§1, chú thích 3). Nói cách khác, ánh xạ D → y phải là 1-1. Benchmark (§3.1): JOB [18] gồm 113 truy vấn từ 33 truy vấn gốc, mỗi gốc có 2–6 biến thể chỉ khác nhau ở filter. STACK [22] gồm 6.191 truy vấn từ 16 truy vấn gốc. Bài không dùng TPC [35] vì TPC giả định dữ liệu phân phối đều.

1.3. Phân tích 4 lỗ hổng trong pipeline ml

1.3.1. Lỗ hổng (1): Sinh dữ liệu huấn luyện (§3)

- Benchmark QOP chỉ cung cấp câu SQL, nên LQO phải tự truy vấn DBMS để lấy đặc trưng và nhãn. Vì vậy dữ liệu dễ mất nhất quán (§1, §3).

Vấn đề	**Giải thích**	**Khuyến nghị**
Thu hẹp không gian kế hoạch (§3.2)	Một số bài tắt nested loop join [18], tắt bitmap/tid scan (Balsa, LEON) hoặc bỏ cây bushy. Cách này giúp vài truy vấn nhưng giảm khả năng tổng quát. Ví dụ: tắt nested loop join làm 16b nhanh hơn nhưng 24b chậm đi [22]. Thí nghiệm của bài: tắt bitmap/tid scan làm 28a nhanh hơn 5,5× nhưng 28b chậm hơn 1,9× (§8.4); bỏ cây bushy làm mất các kế hoạch nhanh nhất (p = 0,015, §8.7)	Không tắt toán tử tùy tiện
Dynamic optimization (§3.3.1)	pg_hint_plan [27] ép kế hoạch, nhưng DBMS có thể đổi kế hoạch khi chạy [1]. Kế hoạch được mã hóa khác kế hoạch thực sự chạy, nên mô hình học trên dữ liệu sai	Dùng direct RL, coi DBMS là hộp đen [8]
Cold và hot cache (§3.3.2)	Cùng một truy vấn chạy lại thì nhanh hơn. Nếu trộn trạng thái cache thì cùng D có nhiều y	Ép hot cache: chạy 3 lần, lấy lần 3 (§7.3). Lần 2 nhanh hơn lần 1 14,6%, sau lần 3 hết xu hướng (§8.6)

1.3.2. Lỗ hổng (2): Mã hóa truy vấn/kế hoạch (§4)

LQO là phương pháp hướng truy vấn [12, 40], nên mã hóa phải vừa bền vững vừa giàu biểu đạt (§4).

- Vi phạm bất biến (§4.1)

    - Bao và Lero không mã hóa truy vấn, chỉ mã hóa kế hoạch. Bao chỉ dùng cardinality và cost, không biết bảng nào nằm ở node nào.
    - Balsa, LEON, HybridQO mã hóa filter chỉ bằng một giá trị cardinality.
    - Hậu quả: các filter khác nhau có thể cho cùng cardinality, nên có cùng vector D nhưng latency khác nhau. Ánh xạ (D, y) trở thành 1-nhiều.
    - Khuyến nghị: dùng embedding (như word2vec của Neo [23]) và vector hóa filter tường minh (như RTOS [42]).
- Thiếu tính biểu đạt (§4.2)
    - Cần cả ngữ cảnh toàn cục (truy vấn) lẫn cục bộ (kế hoạch) [14].
    - Bao và Lero thiếu phần truy vấn nên dễ hội tụ về cực trị địa phương [9].
    - Khuyến nghị: mã hóa cả query lẫn plan.

1.3.3. Lỗ hổng (3): Huấn luyện mô hình (§5)

- Validation sai cách (§5.1)
    - Chỉ RTOS dùng CV, nhưng CV không giúp chọn mô hình cuối.
    - Balsa dừng sớm trên tập validation không cố định.
    - LEON dừng sớm theo accuracy, một chỉ số kém vì ta không biết trước kế hoạch tối ưu.
    - Bao kiểm tra theo kiểu "time series".
    - *Khuyến nghị:* vẫn tinh chỉnh siêu tham số, kể cả với RL; dùng tập holdout cố định [7].
- Đổi biến mục tiêu giữa chừng (§5.2)
    - RTOS, Balsa, Lero, LEON pre-train một mô hình bằng cost rồi chuyển sang latency.
    - Hai đại lượng khác nhau về thang đo và phương sai [2], nên phần lớn kiến thức học lúc pre-train bị mất.
    - *Khuyến nghị:* dùng learning-to-rank [20], hoặc xâu chuỗi nhiều mô hình như HybridQO.
- Bằng chứng (§8.2.2): train lâu hơn không cho kết quả tốt hơn. LEON train 110–130 giờ và Balsa 40–85 giờ nhưng xếp hai vị trí cuối, trong khi Bao chỉ train khoảng 2 giờ.

1.3.4. Lỗ hổng (4): Đánh giá và rò rỉ dữ liệu (§6, §7.2)

Khi cách chia train/test không cố định, kết quả phụ thuộc vào dữ liệu nhiều hơn vào mô hình (data-centric thay vì model-centric) (§1).

Vấn đề	**Giải thích**	**Khuyến nghị**

Tập test thiên lệch (§6.1)	Ext-JOB [23] thêm ORDER BY, làm lợi cho merge join [13]. JOB-Slow [39] chỉ gồm truy vấn ≤ 11 join, đúng vùng PostgreSQL còn vét cạn được. Chia theo số join [43] vô nghĩa vì số join không giải thích được thời gian chạy ($R^2 = -0{,}11$, Hình 2)	Thiết kế các split kiểm tra khả năng tổng quát hóa và dữ liệu ngoài phân phối
Rò rỉ dữ liệu (§7.2)	Các biến thể của cùng một truy vấn gốc có cấu trúc join tối ưu tương tự nhau. Nếu 1a nằm ở tập train còn 1b ở tập test thì thông tin bị rò sang tập test	3 kiểu chia (Hình 3): LOO (dễ), RND (trung bình), BQ (khó, mọi biến thể của một truy vấn gốc nằm cùng một phía)
Bỏ qua thời gian suy luận (§8.2.1)	LEON thực thi chỉ 58 s nhưng suy luận mất khoảng 9,6 giờ	Đo end-to-end: suy luận + lập kế hoạch + thực thi
Covariate shift (§6.2)	Dữ liệu thay đổi sau khi train. Với Bao huấn luyện trên IMDB-50%, truy vấn 31c chậm hơn 24× (§8.3)	Mọi LQO mới nên có thí nghiệm covariate shift

2. Tính SOTA của bài báo

Bài báo không đề xuất phương pháp mới nên không đạt SOTA về hiệu năng. Điểm nổi bật nằm ở cách đánh giá. Nhóm tác giả đánh giá lại các bộ tối ưu truy vấn dựa trên học máy hàng đầu như Neo, Bao, Balsa, LEON và HybridQO [1]–[5] trong cùng một điều kiện. Trước đó, mỗi nghiên cứu thường sử dụng cấu hình hệ quản trị cơ sở dữ liệu, cách chia dữ liệu và cách đo thời gian khác nhau, khiến kết quả khó so sánh trực tiếp.

Để giải quyết vấn đề này, nhóm tác giả xây dựng một khung đánh giá thống nhất về cấu hình hệ quản trị cơ sở dữ liệu, dữ liệu, mã nguồn, cũng như cách chia tập huấn luyện và kiểm thử. Bài báo đưa ra ba cách chia dữ liệu với độ khó tăng dần: giữ lại một biến thể, chia ngẫu nhiên và chia theo truy vấn gốc. Cách chia theo truy vấn gốc giúp tránh rò rỉ thông tin giữa các biến thể của cùng một truy vấn. Thời gian chạy được đo đầy đủ, gồm thời gian suy luận, lập kế hoạch và thực thi truy vấn; thời gian huấn luyện được đo riêng. Ngoài ra, bài báo còn phân tích hai vấn đề trong thiết kế

các bộ tối ưu. Thứ nhất, cách mã hóa truy vấn chỉ dựa trên số lượng bản ghi có thể khiến những truy vấn khác nhau có cùng biểu diễn. Thứ hai, việc chuyển mục tiêu huấn luyện từ chi phí ước lượng sang thời gian thực thi giữa chừng có thể khiến mô hình mất đi những gì đã học trước đó.

Kết quả cho thấy PostgreSQL vẫn đạt hiệu năng tốt nhất trên cả hai bộ dữ liệu JOB và STACK. HybridQO và Bao có kết quả tương đương, đôi khi nhỉnh hơn PostgreSQL ở một số cách chia, nhưng không vượt trội một cách ổn định. Các phương pháp còn lại có kết quả kém hơn đáng kể. Chẳng hạn, riêng bước suy luận của LEON trên tập kiểm thử của JOB đã mất khoảng 9,6 giờ. Đáng chú ý, các phương pháp có thời gian huấn luyện dài hơn lại cho kết quả kém hơn.

So với nghiên cứu cùng hướng của Zhang và cộng sự [6], vốn tập trung đề xuất một giải pháp thay thế đơn giản hơn, bài báo này tập trung vào việc chuẩn hóa quy trình đánh giá và công khai mã nguồn để người khác có thể tái lập kết quả.

Tuy nhiên, bài báo vẫn có một số hạn chế. Thực nghiệm chỉ sử dụng PostgreSQL 12.5, phạm vi truy vấn còn hẹp và mỗi cách chia dữ liệu chỉ có ba bộ chia. Một số phương pháp như Lero, LOGER và RTOS cũng chưa được đưa vào đánh giá.

Tóm lại, bài báo không đề xuất phương pháp mới đạt SOTA về hiệu năng. Đóng góp chính nằm ở việc xây dựng một khung đánh giá thống nhất, giúp so sánh các bộ tối ưu truy vấn dựa trên học máy công bằng và đáng tin cậy hơn.

3. Phát triển từ bài báo (What, How, Why)

3.1. WHAT – Định hướng nghiên cứu

3.1.1. Cơ sở hình thành hướng phát triển

Nghiên cứu của Lehmann và cộng sự [1] chỉ ra rằng nhiều Learned Query Optimizer (LQO) chưa thể hiện ưu thế ổn định so với bộ tối ưu truy vấn của PostgreSQL khi được đánh giá trong một quy trình thống nhất. Kết quả phụ thuộc đáng kể vào cách tạo dữ liệu, mã hóa truy vấn và kế hoạch thực thi, phân chia tập huấn luyện/kiểm thử, cấu hình hệ quản trị cơ sở dữ liệu và phương pháp đo thời gian. Đặc biệt, một kế hoạch truy vấn có thời gian thực thi thấp chưa chắc mang lại thời gian xử lý tổng thể thấp khi chi phí suy luận và lập kế hoạch được tính đến [1, Mục 7–8].

Trên cơ sở đó, hướng phát triển được đề xuất là nghiên cứu một LQO có khả năng lựa chọn kế hoạch thực thi theo tiêu chí hiệu quả đầu cuối, đồng thời được đánh giá trên các nhóm truy vấn chưa xuất hiện trong quá trình huấn luyện. Định hướng này kế thừa các khuyến nghị của bài báo, thay vì xem việc ứng dụng mô hình học máy phức tạp hơn là mục tiêu tự thân.

3.1.2. Mục tiêu nghiên cứu

- Giảm tổng thời gian xử lý truy vấn (end-to-end latency), gồm thời gian suy luận của LQO, thời gian lập kế hoạch của DBMS và thời gian thực thi SQL.
- Cải thiện khả năng tổng quát hóa đối với những mẫu cấu trúc truy vấn chưa gặp trong huấn luyện; đồng thời xem xét độ bền vững khi phân bố dữ liệu thay đổi.
- Đề xuất quy trình đánh giá có khả năng tái lập, đối chiếu được với PostgreSQL trong điều kiện phần cứng, cấu hình và dữ liệu thống nhất.

3.1.3. Phạm vi của đề xuất

Nội dung dưới đây là thiết kế ở mức định hướng và quy trình thực nghiệm dự kiến. Bài báo [1] cung cấp phân tích và các khuyến nghị riêng lẻ, nhưng không xây dựng hoặc xác nhận hiệu quả của chính kiến trúc kết hợp được đề xuất trong báo cáo này. Do đó, mọi nhận định về lợi ích của kiến trúc mới cần được hiểu là giả thuyết nghiên cứu cần kiểm chứng.

3.2. HOW – Khuyến nghị và giải pháp kỹ thuật

3.2.1. Kiến trúc tổng quan được đề xuất

Đề xuất sử dụng cách tiếp cận xếp hạng kế hoạch truy vấn: từ một câu SQL, hệ thống tạo tập kế hoạch ứng viên, biểu diễn thông tin của câu truy vấn và từng kế hoạch, sau đó dùng mô hình Learning-to-Rank (LTR) để sắp xếp các ứng viên. Kế hoạch được ưu tiên sẽ chuyển cho PostgreSQL thực thi. Quá trình huấn luyện và đánh giá sử dụng quy tắc phân chia truy vấn theo nhóm gốc, đo thời gian với trạng thái cache nhất quán và cấu hình DBMS được kiểm soát.

Thành phần	**Vai trò trong giải pháp**	**Cơ sở từ bài báo**
Query & Plan Encoding	Kết hợp bối cảnh của câu SQL và đặc trưng của kế hoạch thực thi trong đầu vào mô hình.	[1, Mục 4]
Learning-to-Rank (LTR)	Học thứ tự ưu tiên giữa các kế hoạch ứng viên dựa trên nhãn so sánh hiệu năng.	[1, Mục 2; 5.2]
Base Query Sampling	Đánh giá trên các họ truy vấn chưa xuất hiện trong dữ liệu huấn luyện.	[1, Mục 7.2]
Hot Cache	Chuẩn hóa điều kiện đo thời gian truy vấn, hạn chế sai lệch do cache.	[1, Mục 3.3.2; 7.3; 8.6]

DB Tuning	Chuẩn hóa cấu hình PostgreSQL và duy trì baseline đủ mạnh.	[1, Mục 7.1; 8.4–8.5]

3.2.2. Biểu diễn đồng thời Query và Plan

Đối với phần query encoding, mô hình nên tiếp nhận thông tin thể hiện cấu trúc quan hệ giữa các bảng, điều kiện lọc và các đặc trưng cần thiết của truy vấn. Đối với plan encoding, mô hình cần thể hiện cấu trúc cây thực thi và các toán tử vật lý như join và scan. Việc sử dụng cả hai nguồn thông tin nhằm giảm rủi ro những truy vấn hoặc kế hoạch khác nhau bị biểu diễn quá giống nhau, đồng thời giữ được bối cảnh tổng thể và cục bộ [1, Mục 4].

Bài báo lưu ý rằng việc chỉ sử dụng các giá trị ước lượng cardinality có thể gây trùng lặp biểu diễn giữa các trường hợp khác nhau. Do đó, phương án triển khai cần kiểm tra độ phân biệt của các đặc trưng và cân nhắc biểu diễn vector cho điều kiện lọc hoặc thuộc tính văn bản khi phù hợp. Đây là yêu cầu thiết kế đầu vào, không phải cam kết rằng một phương pháp encoding cụ thể chắc chắn cải thiện hiệu năng.

3.2.3. Xếp hạng kế hoạch bằng Learning-to-Rank

LTR được đề xuất để so sánh tương đối các kế hoạch ứng viên của cùng một truy vấn, thay vì chỉ dự đoán một giá trị thời gian tuyệt đối. Theo phân tích tại Mục 5.2 của [1], cách học theo thứ hạng có ưu điểm tiềm năng khi sử dụng cả nhãn cost và latency, bởi các giá trị có thể được chuyển thành quan hệ thứ tự. Tuy nhiên, LTR không tự động loại bỏ chi phí sinh kế hoạch ứng viên, suy luận hoặc sai số xếp hạng.

Quy trình dự kiến gồm: (i) tạo các kế hoạch ứng viên trong phạm vi ngân sách tính toán xác định; (ii) mã hóa cặp truy vấn–kế hoạch; (iii) lấy thông tin cost và, đối với tập phù hợp, thời gian thực thi được đo nhất quán; (iv) huấn luyện mô hình xếp hạng; và (v) lựa chọn ứng viên dự kiến có hiệu quả tốt. Khi đánh giá, tiêu chí cuối cùng vẫn là end-to-end latency, không chỉ độ chính xác của thứ hạng.

3.2.4. Phân chia dữ liệu bằng Base Query Sampling

Trong các workload gồm nhiều biến thể cùng một mẫu SQL, chia ngẫu nhiên có thể đưa những truy vấn rất giống nhau vào cả tập train lẫn test. Bài báo [1, Mục 7.2] đề xuất Base Query Sampling, trong đó toàn bộ biến thể thuộc cùng một base query chỉ xuất hiện ở một phía của phép chia. Cách tiếp cận này tạo bài kiểm tra khó hơn và phản ánh tốt hơn khả năng xử lý các cấu trúc truy vấn chưa được học.

Đề xuất sử dụng Base Query Sampling làm phép thử trọng tâm, đồng thời báo cáo thêm kết quả Random Sampling và Leave-One-Out để nhận biết độ nhạy của mô hình đối với cách chia tập dữ liệu. Các lần so sánh phải dùng cùng tập train/test cho tất cả phương pháp.

3.2.5. Chuẩn hóa điều kiện đo: Hot Cache và DB Tuning

Để hạn chế dao động từ bộ nhớ đệm, mỗi truy vấn được thực thi lặp lại trong điều kiện nhất quán. Trong thí nghiệm của [1, Mục 8.6], lần chạy thứ ba là lựa chọn cân bằng giữa độ ổn định và chi phí đo; vì vậy có thể áp dụng quy tắc chạy ba lần và lấy kết quả lần thứ ba theo thiết lập hot cache. Khi thay đổi hạ tầng hoặc workload, quy tắc này vẫn cần được kiểm tra lại.

Cấu hình PostgreSQL cũng cần được ghi nhận và cố định, bao gồm các tham số bộ nhớ, song song hóa, join/scan và GEQO. Không nên tùy tiện vô hiệu hóa bitmap scan, tid scan hoặc GEQO để làm đơn giản không gian tìm kiếm, vì [1, Mục 8.4–8.5] cho thấy thao tác này có thể cải thiện một số truy vấn nhưng làm chậm những truy vấn khác. Baseline PostgreSQL phải được đánh giá trong điều kiện tương đương và cấu hình hợp lý.

3.2.6. Kế hoạch kiểm chứng đề xuất

Nhóm đánh giá	**Chỉ số / cách kiểm chứng**
Hiệu quả đầu cuối	Đo inference time, planning time, execution time và tổng ba thành phần; so sánh với PostgreSQL.
Tổng quát hóa	So sánh hiệu năng giữa Leave-One-Out, Random và Base Query Sampling trên cùng quy ước chia tập.
Ổn định và khả năng tái lập	Giữ cố định tập truy vấn, cấu hình DBMS, trạng thái cache, môi trường và cách đo.
Khả năng thích ứng dữ liệu	Thử nghiệm thay đổi quy mô/phân bố dữ liệu, đánh giá suy giảm hiệu năng và nhu cầu huấn luyện lại.
Chi phí triển khai	Ghi nhận thời gian tạo dữ liệu, huấn luyện, suy luận và số lượng kế hoạch ứng viên cần xét.

JOB và STACK là hai workload phù hợp để thiết kế thí nghiệm đối chiếu với [1]. Việc lựa chọn mô hình LTR cụ thể, số lượng ứng viên, siêu tham số và ngân sách tính toán là các quyết định thực nghiệm cần được xác lập trước khi tiến hành chạy thử; báo cáo hiện tại chưa đưa ra kết quả đo cho kiến trúc đề xuất.

3.3. WHY – Cơ sở và ý nghĩa của hướng phát triển

3.3.1. Tối ưu hóa phải xét toàn bộ chi phí xử lý

Mục tiêu của tối ưu truy vấn là giảm thời gian người dùng phải chờ kết quả, không chỉ giảm phần thời gian SQL thực thi. Bài báo cho thấy một số mô hình tìm được kế hoạch khả quan nhưng phải trả giá bằng thời gian suy luận lớn; trường hợp LEON là ví dụ đáng chú ý [1, Mục 8.2.1]. Vì vậy, lấy end-to-end latency làm tiêu chí chính giúp gắn đánh giá nghiên cứu với lợi ích sử dụng thực tế.

3.3.2. Khả năng tổng quát hóa quyết định tính hữu dụng

Môi trường cơ sở dữ liệu thực tế không giới hạn ở các truy vấn đã xuất hiện trong huấn luyện. Những biến thể từ cùng một mẫu SQL có thể làm kết quả kiểm thử lạc quan nếu không kiểm soát sự tương đồng giữa train và test. Bên cạnh đó, dữ liệu có thể tăng hoặc thay đổi theo thời gian. Bài báo quan sát thấy hiệu năng của mô hình có thể suy giảm khi trạng thái dữ liệu thay đổi, dù PostgreSQL đã cập nhật thống kê [1, Mục 6.2; 8.3]. Vì vậy, cần thử nghiệm khả năng tổng quát hóa và cân nhắc cơ chế cập nhật mô hình.

3.3.3. So sánh công bằng là điều kiện để kết luận có giá trị

Khác biệt về cache, cấu hình DBMS và cách chia dữ liệu có thể làm thay đổi thứ hạng các phương pháp. Nếu không chuẩn hóa môi trường, cải thiện được báo cáo có thể phản ánh điều kiện thí nghiệm thuận lợi hơn là chất lượng của thuật toán. Việc kế thừa benchmark thống nhất từ [1] giúp phân biệt đóng góp thực sự của mô hình mới với tác động của thiết lập thực nghiệm.

3.4. Kết luận và hướng nghiên cứu tiếp theo

Từ những hạn chế được chỉ ra trong [1], báo cáo đề xuất định hướng LQO dựa trên biểu diễn đồng thời query và plan, học xếp hạng các kế hoạch ứng viên, và lấy tổng thời gian xử lý đầu cuối làm tiêu chí lựa chọn và đánh giá. Base Query Sampling, quy trình đo hot cache và cấu hình PostgreSQL nhất quán được sử dụng như các biện pháp kiểm chứng khả năng tổng quát hóa và tính công bằng của kết quả.

Giá trị kỳ vọng của hướng nghiên cứu là một LQO có khả năng mang lại cải thiện ổn định, chi phí suy luận chấp nhận được và kết quả có thể tái lập. Tuy nhiên, đây mới là đề xuất kỹ thuật. Để kết luận mô hình có ưu thế so với PostgreSQL, cần triển khai nguyên mẫu và tiến hành thí nghiệm trên JOB, STACK với các tập chia và điều kiện đo được công bố rõ ràng.

4. Kế hoạch thực nghiệm và mở rộng

4.1. Mục tiêu

- Tái lập kết luận chính của bài báo trên một nền tảng phần cứng khác: PostgreSQL nhìn chung vẫn tốt nhất (§8.2.1).
- Đo plan mới nhanh hơn bao nhiêu: với mỗi câu truy vấn, chỉ ra plan cũ (của PostgreSQL), plan tối ưu và mức tăng tốc.
- Phân loại plan bằng data mining: dự đoán mỗi câu nên dùng plan nào và đếm số câu dự đoán đúng, để chứng minh bằng thuật toán thay vì liệt kê từng câu.

Nhóm dùng mã nguồn và log mà tác giả công bố tại github.com/edualc/lqo_ml_perspective.

4.2. Hệ quản trị CSDL và môi trường

Thành phần	**Bài báo (§8.1.1, Bảng 2)**	**Nhóm**
Phần cứng	x86, 16 CPU, 64 GB RAM, GPU T4	MacBook M1 Pro (ARM64), 8 CPU, 16 GB RAM, không GPU
DBMS	PostgreSQL 12.5 + pg_hint_plan	Giống bài báo, build từ mã nguồn cho ARM64
shared_buffers / work_mem / effective_cache_size	32 GB / 4 GB / 32 GB	3 GB / 128 MB / 8 GB
GEQO, bitmap/tid scan	Bật	Bật
autovacuum	Tắt, ANALYZE một lần	Giống bài báo
Đo thời gian	Chạy 3 lần, lấy lần thứ 3 (hot cache); timeout 3 phút	Giống bài báo

Lưu ý về hệ quản trị CSDL:

- Phải dùng đúng PostgreSQL 12.5, vì pg_hint_plan và pg_bao chỉ hỗ trợ phiên bản này.

- Bộ nhớ phải thu nhỏ cho vừa 16 GB RAM. Vì vậy nhóm chỉ so sánh xu hướng và thứ hạng, không so sánh số tuyệt đối (§7.1).
- Giữ nguyên GEQO, bitmap/tid scan và autovacuum như bài báo, vì bài báo chứng minh thay đổi các thiết lập này làm thay đổi đáng kể kết quả (§8.4, §8.5).

Triển khai:

- Hệ thống gồm 2 container giống repo gốc: pg_bao (PostgreSQL) và bao_server (Bao, PyTorch).
- Để chạy trên ARM64, nhóm build PostgreSQL và các extension từ mã nguồn, đổi PyTorch sang bản aarch64, và sửa đường dẫn tải IMDB cùng tên CSDL trong bao.cfg.
- Dữ liệu IMDB nạp xong gồm 21 bảng, dung lượng 12 GB.

4.3. Phương pháp chạy lại

Phương pháp	**Yêu cầu (§8.2.2)**	**Kế hoạch**
PostgreSQL	Không cần train	Chạy 113 câu JOB × 3 lần
Bao	CPU, khoảng 2 giờ/split	Chạy trên 3 split: LOO-1, RND-1, BQ-1
HybridQO, Neo, Balsa, LEON	20–130 giờ/split, phần lớn cần GPU	Dùng log của tác giả

Nhóm chạy thêm các ablation không cần GPU: bitmap/tid scan, GEQO, số lần chạy k, cây bushy, covariate shift (§8.3–§8.7).

Cách so sánh: một câu là thắng nếu nhanh hơn PostgreSQL trên 10%, thua nếu chậm hơn trên 10%, còn lại là hòa. Kiểm định bằng Wilcoxon và sign test.

4.4. Metric đo câu SQL

Loại	**Metric**
Tĩnh (đo độ phức tạp trước khi chạy)	Số bảng, số join, số filter, số LIKE/IN/OR/BETWEEN; số node và độ sâu của cây plan; có dùng cây bushy hay không; số Hash/Merge/Nested Loop join; số Seq/Index/Bitmap scan; cost và số dòng ước lượng
Động (đo khi chạy)	Planning time, execution time, end-to-end time (metric chính, §8.2.1); speedup =

	t_cũ / t_mới; % thời gian tiết kiệm; q-error = max(ước lượng/thực tế, thực tế/ước lượng); số câu timeout; tổng thời gian workload

4.5. Chạy plan và lưu log

Các plan được so sánh: mỗi câu được chạy với 6 hint set kiểu Bao (§2):

- **default:** plan cũ, do PostgreSQL tự chọn**.**
- **no_nestloop, no_hashjoin, no_mergejoin, no_seqscan, no_indexscan:** mỗi hint set tắt một loại toán tử bằng lệnh SET enable_... = off, buộc PostgreSQL chọn một plan khác.

Tổng cộng khoảng 113 × 6 × 3 ≈ 2.000 lần chạy. Plan tối ưu của một câu là hint set có end-to-end time nhỏ nhất.

Định dạng log: mỗi lần chạy được ghi thành một dòng JSON, kèm toàn bộ plan lấy từ EXPLAIN (ANALYZE, FORMAT JSON):

```
{"query_id": "1a", "db": "imdbload", "pg_version": "12.5", "hint_set": "default",
"run": 2, "planning_ms": ..., "execution_ms": ..., "timeout": false, "plan": {...}}
{"query_id": "1a", "db": "imdbload", "pg_version": "12.5", "hint_set":
"no_nestloop", "run": 2, "planning_ms": ..., "execution_ms": ..., "timeout": false,
"plan": {...}}
{"query_id": "1b", "db": "imdbload", "pg_version": "12.5", "hint_set": "default",
"run": 2, "planning_ms": ..., "execution_ms": ..., "timeout": false, "plan": {...}}
```

- query_id: phần số (1) là truy vấn gốc, dùng khi chia dữ liệu theo Base Query.
- run = 2: lần chạy thứ 3 (hot cache), lần được dùng để phân tích.
- plan: cây plan đầy đủ của lần chạy đó.

Trình bày plan thực thi: từ trường plan, nhóm trích ra loại toán tử, bảng, thứ tự join, cost, số dòng ước lượng và số dòng thực tế, rồi đặt plan cũ cạnh plan mới. Ví dụ minh họa cách trình bày với câu 1a (chưa phải kết quả đo):

```
Plan cũ (default)                          Plan mới (no_nestloop)
Aggregate                                  Aggregate
└─ Nested Loop                             └─ Hash Join
   ├─ Nested Loop                             ├─ Seq Scan on title t
   │  ├─ Seq Scan on info_type it             └─ Hash Join
   │  └─ Index Scan on movie_info_idx            ├─ Hash Join (mi_idx ⋈ it)
   └─ Index Scan on title t ...                  └─ Hash Join (mc ⋈ ct)
```

Hai plan được so sánh theo 4 điểm: thứ tự join, loại join/scan, hình dạng cây (left-deep hay bushy), và khoảng cách giữa hai cây (tree edit distance).

Bảng plan tối ưu cho toàn bộ dataset. Mỗi câu một dòng, gồm các cột: query_id, plan cũ, plan tối ưu, thời gian cũ, thời gian mới, speedup. Từ bảng này nhóm báo cáo:

- Số câu có plan tốt hơn plan mặc định.
- Phân phối speedup: trung vị, lớn nhất, số câu nhanh hơn 2×.
- Tổng thời gian workload khi luôn dùng plan cũ so với khi luôn dùng plan tối ưu.

4.6. Phân loại plan bằng data mining

Bài toán

- Đầu vào: metric tĩnh của câu SQL và của plan mặc định. Đây là thông tin có sẵn trước khi thử các plan khác, nên không rò rỉ nhãn.
- Nhãn: plan tối ưu. Một hint set chỉ được tính là tốt hơn default nếu nhanh hơn ít nhất 5%.
- Đầu ra: mỗi câu được gán một trong 6 plan. Ví dụ với 1.000 câu, mô hình gán cho mỗi câu một plan.

Chia dữ liệu theo đúng 3 kiểu của bài báo (§7.2):

- Leave One Out (dễ).
- Random (KFold, trung bình).
- Base Query (GroupKFold theo truy vấn gốc, khó nhất, không rò rỉ giữa các biến thể).

Mô hình: Decision Tree, Random Forest, Gradient Boosting, kNN, Naive Bayes, SVM, Logistic Regression. So sánh với hai mốc:

- "Luôn dùng PostgreSQL": không bao giờ đổi plan.

- "Oracle": luôn chọn đúng plan tối ưu.

Đánh giá:

- Số câu dự đoán đúng / tổng số câu (ví dụ: 80/113), kèm ma trận nhầm lẫn.
- Accuracy và macro-F1.
- Tổng thời gian workload khi dùng plan do mô hình chọn, so với hai mốc ở trên.
- Số câu bị chậm đi trên 10%.
- Kiểm định McNemar.

Nhóm ưu tiên các chỉ số theo thời gian, vì bài báo chỉ ra accuracy không phù hợp cho tối ưu truy vấn (§5.1).

Giải thích bằng thuật toán:

Thuật toán	**Mục đích**
K-means + PCA	Tìm các nhóm câu truy vấn thường được lợi khi đổi plan
Apriori	Tìm luật {đặc điểm câu/plan} → plan tốt nhất (support, confidence, lift)
Permutation importance	Xác định metric quyết định việc nên đổi plan, ví dụ q-error
Cây quyết định độ sâu 3	Đưa ra quy tắc ngắn gọn, dễ đọc
Tree edit distance + Spearman	Kiểm tra plan mới khác plan cũ càng nhiều thì có càng nhanh không

4.7. Thêm benchmark

JOB chỉ có 113 câu, quá ít cho bài toán phân loại. Nhóm đề xuất thêm CEB-IMDb:

- Chạy trên cùng CSDL IMDB nên không cần nạp dữ liệu mới.
- Nhóm lấy mẫu khoảng 1.000 câu và lặp lại mục 4.5–4.6.
- Phương án dự phòng: DSB.
- Không dùng STACK vì dữ liệu quá lớn.

4.8. Kết quả sơ bộ trên log của tác giả

Nhóm phân tích lại log của tác giả trên 9 split JOB (chỉ tập test), so sánh từng câu với PostgreSQL:

Phương pháp	**Thắng**	**Hòa**	**Thua**	**Speedup tổng**

HybridQO	34	82	117	1,00×
Bao	9	34	190	0,83×
Balsa	16	4	213	0,28×
Neo	21	12	200	0,16×
LEON	0	0	233	0,001×

Nhận xét:

- Thứ hạng khớp với bài báo (§8.2.1).
- HybridQO ngang PostgreSQL về tổng thời gian nhưng thua ở khoảng một nửa số câu. Điều này cho thấy mỗi LQO chỉ có lợi ở một số câu nhất định.
- Đây chính là động lực cho phần phân loại plan: nếu dự đoán trước được câu nào nên đổi plan, có thể giữ phần lợi và tránh phần thiệt.

4.9. Kết luận và hạn chế

Nhóm đã dựng xong môi trường PostgreSQL 12.5 trên ARM64 với cấu hình bám sát bài báo. Phân tích lại log của tác giả xác nhận không có LQO nào vượt PostgreSQL một cách ổn định. Bước tiếp theo là chạy 6 plan cho mỗi câu, đo mức tăng tốc, rồi dùng phân loại và data mining để dự đoán plan phù hợp cho từng câu trên JOB và CEB-IMDb.

Hạn chế:

- Chỉ chạy lại PostgreSQL và Bao; các LQO khác dùng log của tác giả.
- Mốc tối ưu chỉ là plan tốt nhất trong 6 hint set, chưa phải plan tối ưu toàn cục.

Tài liệu tham khảo

[1] R. Marcus et al., "Neo: A Learned Query Optimizer," *PVLDB*, vol. 12, no. 11, 2019.
[2] R. Marcus et al., "Bao: Making Learned Query Optimization Practical," in *Proc. SIGMOD*, 2021, pp. 1275–1288.
[3] Z. Yang et al., "Balsa: Learning a Query Optimizer Without Expert Demonstrations," in *Proc. SIGMOD*, 2022, pp. 931–944.
[4] X. Chen et al., "LEON: A New Framework for ML-Aided Query Optimization," *PVLDB*, vol. 16, no. 9, pp. 2261–2273, 2023.
[5] X. Yu et al., "Cost-based or Learning-based? A Hybrid Query Optimizer for Query Plan Selection," *PVLDB*, vol. 15, no. 13, pp. 3924–3936, 2022.
[6] Y. Zhang et al., "Simple Adaptive Query Processing vs. Learned Query Optimizers: Observations and Analysis," *PVLDB*, vol. 16, no. 9, pp. 2962–2975, 2023.